

CVNet 1.0

User's Manual

Guanghong Zuo
ghzuo@ucas.ac.cn

May 19, 2026

Contents

1	Introduction	1
2	CVNet Working Model	2
2.1	Constructing Composition Vectors	3
2.2	Similarity Algorithms	4
2.3	Network Construction Algorithms	5
3	Composition Vector Acceleration Algorithms	6
4	Object-Oriented Software Design	8
5	Basic Usage	9
	Reference	10

1 Introduction

Orthologous genes refer to gene pairs derived from different species that have been directly inherited vertically from a common ancestral gene through speciation events (Fitch, 1970). As "functional messengers" in genome evolution, they typically retain their original molecular functions during the evolutionary process, thus occupying a central position in comparative genomics, functional genomics, and phylogenetics. By accurately identifying orthologous genes, researchers can infer gene functions, trace the evolutionary history of species, reveal the expansion and contraction of gene families, and understand the molecular mechanisms driving biodiversity (Koonin, 2005). Therefore, the precise inference of orthologous relationships is the cornerstone of many bioinformatics analyses, and its accuracy directly affects the reliability of downstream analysis conclusions.

Currently, computational methods for identifying orthologous genes can be mainly categorized into several types. The first category is based on the "Reciprocal Best Hit" (RBH) method, such as derivatives of the BLAST algorithm, which identify gene pairs with mutual optimal matches between two species through pairwise genome comparison (Tatusov et al., 1997; Altschul et al., 1990). These methods are computationally simple and efficient but tend to misclassify paralogous genes as orthologous, especially within gene families that have undergone gene duplication events. The second category involves phylogenetic tree-based methods, which infer orthology by constructing and comparing gene trees and species trees across multiple species (Eisen, 1998; Zmasek and Eddy, 2001). While theoretically the most rigorous, this approach incurs high computational costs and relies heavily on the accuracy of multiple sequence alignment and tree-building algorithms. To balance efficiency and precision, a third category—graph-based clustering methods—has emerged and gained wide application, such as OrthoMCL (Li et al., 2003) and OrthoFinder (Emms and Kelly, 2015). These methods connect gene sequences from entire genomes into a graph based on similarity and then use clustering algorithms (e.g., Markov Clustering Algorithm, MCL) to partition potential orthologs and paralogs into distinct clusters. Despite the success of existing methods, they still face challenges in handling large-scale genomic data, complex evolutionary scenarios (such as incomplete lineage sorting), and computational resource consumption.

In our preliminary research, we focused on the Composition Vector Tree (CVTree) method for microbial evolution and taxonomy (Zuo, 2021; Lu et al., 2025). This method was creatively proposed by Professor Hao Bailin and coworkers as an approach for tree construction using whole-genome data (Qi et al., 2004a,b; Xu and Hao, 2009; Zuo, 2021; Zuo and Hao, 2015). Essentially, the CVTree method is a distance matrix method. Its core idea is based on statistical analysis of short strings of specific lengths within genomes. These statistics are transformed via a mathematical model to represent each genome as a high-dimensional vec-

tor. Distances between these vectors are then calculated, and the Neighbor-Joining method is used for tree construction. This method is highly efficient and capable of processing the evolution and classification of tens of thousands of genomes. Fundamentally, the phylogenetic relationship between species and the homology between genes are both manifestations of species evolution. In other words, the principles and methods of CVTree are equally applicable to the study of gene homology. Moreover, the efficiency of CVTree suggests that, based on this method, a more efficient tool for homology searching can be developed to accommodate the current surge in gene sequence data. Consequently, building upon the original CVTree software, we developed CVNet, a tool for identifying orthologous genes. The workflow of CVNet is illustrated in Figure 1. Methodologically, CVNet belongs to the third category mentioned above, i.e., graph-based clustering methods. It first transforms genes into vectors based on the CVTree statistical model, calculates similarities between genes based on these vectors to construct a homology network, and finally employs a graph clustering algorithm to obtain orthologous families. During development, besides utilizing the foundational models, CVNet introduced new data structures tailored to the problem and implemented parallelism based on OpenMP to enhance software performance. Additionally, we emphasized software openness during development, designing open interfaces at each step of program execution using object-oriented methods, allowing for convenient integration of new mathematical models.

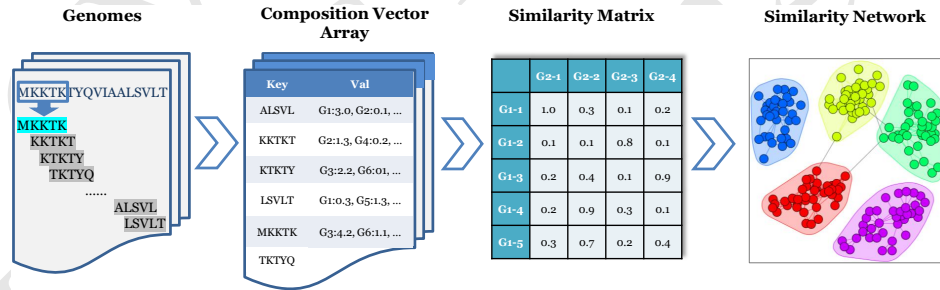


Figure 1: CVNet Workflow

2 CVNet Working Model

The steps of the CVNet model are shown in Figure 2. First, it can process different gene sequences, including protein and DNA sequences. Second, when converting each genome into a composition vector, besides selecting different lengths (K), two available models can be used: the Count model and the Hao model. Third, there are six different options for calculating similarity. In the third step, from the similarity matrix to the connection network, we provide four optional schemes. Finally, the process from the connection network to

orthologous groups strictly speaking falls outside the CVNet code itself and is implemented by calling the Markov Clustering (MCL) program.

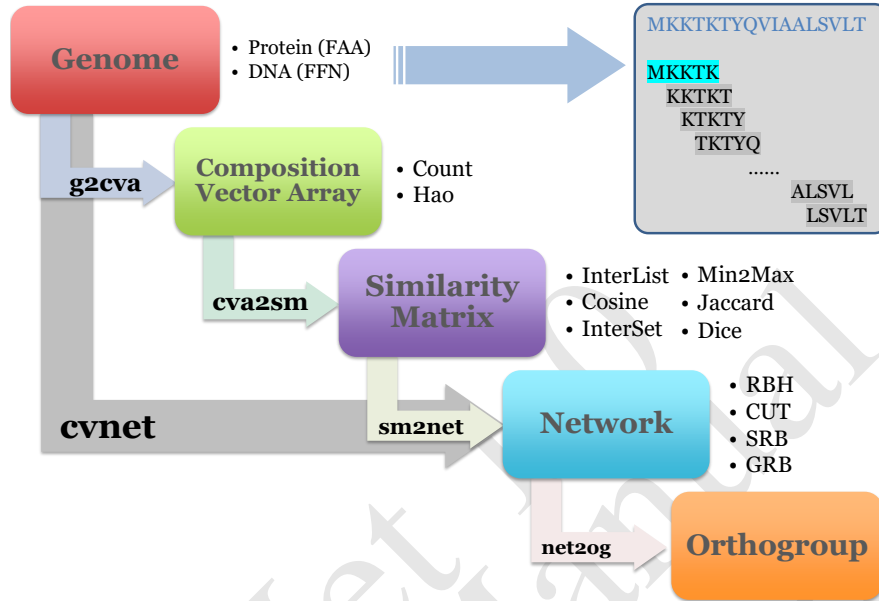


Figure 2: CVNet Model Steps

2.1 Constructing Composition Vectors

Regarding the construction of composition vectors, we follow the settings in CVTree, which include two methods: the Count method and the Hao method. The Count method is relatively straightforward. As shown in the upper right corner of Figure 2, for each gene, a window of length K slides one residue at a time from beginning to end, counting the frequency of occurrence of various K -mers, denoted as $f(a_1 a_2 \cdots a_K)$, where a_i represents an amino acid residue. The probability of occurrence of this K -mer is:

$$p(a_1 a_2 \cdots a_K) = \frac{f(a_1 a_2 \cdots a_K)}{N_K}$$

where $N_K = L - K + 1$ is the total number of K -mers obtained. Thus, we can use the string $a_1 a_2 \cdots a_K$ as the label for a dimension, and its count or probability as the value for that dimension, forming a composition vector. That is, depending on the chosen K value, the dimensionality of the composition vector space increases exponentially. For example, for protein sequences with 20 amino acids, when $K = 5$, the dimensionality of the space is 20^5 .

The Count model may underperform in certain situations. However, these statistics can

be further mapped to another space via a model to improve results. The Hao model in CVNet is a model we developed during our research on species evolution. The basic idea stems from Motoo Kimura's neutral theory of evolution. Kimura's theory posits that genomes retain a large number of "not-so-good, not-so-bad" neutral mutations. The contribution of these mutations to the aforementioned K -mer counts is unrelated to the speciation process and should be treated as background noise to be subtracted. Therefore, based on a Markov model, we use the probabilities of $(K - 1)$ -mers and $(K - 2)$ -mers to predict the noise of gene mutations, i.e.:

$$\tilde{p}(a_1 a_2 \cdots a_K) = \frac{p(a_2 a_3 \cdots a_{K-1}) p(a_1 a_2 \cdots a_{K-1})}{p(a_2 a_3 \cdots a_{K-1})}$$

Here, $\tilde{p}$ denotes the estimated probability of the K -mer given by the probabilities of $(K - 1)$ - and $(K - 2)$ -strings, i.e., the noise. We take the deviation between the directly counted frequency f of the K -mer and its estimated value f^0 as the composition vector of the gene:

$$v(a_1 a_2 \cdots a_K) = \frac{p(a_1 a_2 \cdots a_{K-1}) - \tilde{p}(a_1 a_2 \cdots a_K)}{\tilde{p}(a_1 a_2 \cdots a_K)}$$

For N genes, we obtain N such composition vectors V_i , where i is the species index ranging from 1 to N .

2.2 Similarity Algorithms

After obtaining the composition vectors representing the genes, we can calculate the similarity between genomes based on these vectors. In the current version of CVNet, there are six methods:

1. Cosine:

$$d_{ij} = \frac{1}{2} \left(1 + \frac{\vec{V}_i \cdot \vec{V}_j}{\|\vec{V}_i\|_2 \cdot \|\vec{V}_j\|_2} \right)$$

2. InterList:

$$d_{ij} = \frac{2.0 \cdot \|\min \circ (\vec{V}_i, \vec{V}_j)\|_1}{\|\vec{V}_i\|_1 + \|\vec{V}_j\|_1}$$

3. Min2Max:

$$d_{ij} = \frac{2.0 \cdot \|\min \circ (\vec{V}_i, \vec{V}_j)\|_1}{\|\vec{V}_i\|_1 + \|\vec{V}_j\|_1}$$

4. InterSet:

$$d_{ij} = \frac{\|\min \circ (\vec{V}_i, \vec{V}_j)\|_1}{\|\max \circ (\vec{V}_i, \vec{V}_j)\|_1}$$

5. Jaccard:

$$d_{ij} = \frac{\|\vec{V}_i \odot \vec{V}_j\|_0}{\sqrt{\|\vec{V}_i\|_0 \cdot \|\vec{V}_j\|_0}}$$

6. Dice:

$$d_{ij} = \frac{\|\vec{V}_i \odot \vec{V}_j\|_0}{\|\vec{V}_i + \vec{V}_j\|_0} \quad \text{or} \quad d_{ij} = \frac{2 \cdot \|\vec{V}_i \odot \vec{V}_j\|_0}{\|\vec{V}_i\|_0 + \|\vec{V}_j\|_0}$$

The double vertical bars denote norms, and the subscript indicates the norm type (L_2, L_1, L_0). The similarity values calculated by the above formulas range from 0 to 1, where 1 indicates identical and 0 indicates completely different. Thus, for N genes, the similarities form an $N \times N$ symmetric matrix with diagonal elements equal to 1, which we call the similarity matrix.

2.3 Network Construction Algorithms

The aforementioned similarity matrix itself can be transformed into a fully connected weighted matrix. However, directly inputting this matrix into the MCL clustering algorithm or other clustering algorithms usually fails to effectively identify orthologous genes. Moreover, due to the presence of numerous near-zero elements, computation time can be excessively long, potentially leading to failure in obtaining results. Therefore, truncation processing is required before input to the MCL program to remove some network connections. In general, there are four processing rules:

1. RBH (Reciprocal Best Hit): Find gene pairs with mutual optimal matches between two species. In matrix operations, find elements that are maximum in both their row and column, connect the genes corresponding to their row and column indices, and build the network as input for the subsequent MCL program.

2. CUT (Cutoff): Relatively simple; retain elements in the matrix greater than a specific threshold, connect the genes corresponding to these retained elements, and build the network as input for the subsequent MCL program.

3. SRB (Sequence Reciprocal Best): This method first finds all mutually optimal matching gene pairs between two sequences (genomes). Then, it identifies the minimum similarity

value among these pairs as the threshold for preserving gene similarities between the two genomes.

4. GRB (Gene Reciprocal Best): This method also requires finding all mutually best matching gene pairs first, but unlike the previous method, it focuses on specific genes. For each gene as a group, it finds all mutually best matching gene pairs containing that gene, obtains the minimum similarity among them as a threshold, retains all gene pairs with similarity greater than this threshold for that gene, and connects them into a network as input for the subsequent MCL program. Research indicates that this method might be the optimal approach for network construction in the current CVNet program.

3 Composition Vector Acceleration Algorithms

The establishment of mathematical models provides a foundation for solving problems. To utilize computers for problem-solving, we further design appropriate computational algorithms for the models to achieve efficient solutions. The core issue of computational algorithms is how to efficiently organize and index data for specific problems. In the CVNet algorithm design, three algorithms are noteworthy: K-string encoding, the CVArray data structure, and the ordered sequence alignment algorithm.

In the CVNet model, each species is represented as a CV vector through vectorization, where each dimension's K -mer label represents a specific feature of the data. In traditional representations, bases in DNA sequences and residues in protein sequences are represented by single letters; for example, the letter "A" represents adenine in DNA and alanine in protein sequences. Therefore, if each K -mer is represented as a string array in a computer, it would severely impact indexing and storage efficiency. Thus, in CVNet, each letter is numbered and used as a digit. Consequently, each K -mer is represented as a base- N K -digit number. For instance, for nucleic acid sequences, the letters A, T, C, G can be represented as 0, 1, 2, 3. Thus, the K -mer "GTACA" is represented by the quaternary number "31020", which converts to decimal 840. This allows us to represent and manipulate string data using a long integer, enabling more efficient retrieval and storage (as shown in Figure 3, a Kmer, i.e., a K -mer, is an 8-byte long integer). Additionally, there is an improved encoding method that uses 0 for gap encoding, which enhances encoding distinguishability (e.g., distinguishing K -mers of different lengths) and improves scalability, albeit at the cost of wasting some integer representation space.

As mentioned earlier, each dimension of the vector represents a string of length K . Taking protein sequences as an example, the dimensionality is 20^K . Assuming $K = 5$, each species would require a vector of dimension 64×10^5 for representation. However, the to-

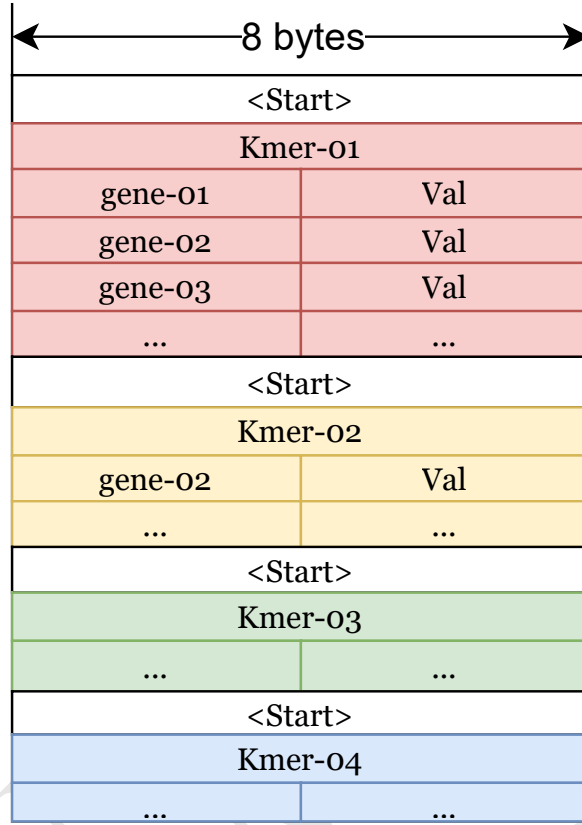


Figure 3: K-string Encoding and CVArray

tal length of protein sequences for a typical prokaryotic species is generally on the order of 10^6 , and even without considering repeated K -mers, it is impossible to generate that many K -mers. In other words, many dimensions have zero counts; it is a sparse vector. Therefore, during computation and storage, only non-zero dimensions are stored as key-value pairs. However, this necessitates comparing keys one by one during later similarity calculations, incurring significant computational overhead. Considering that each genome participates as a whole in every calculation, we merge the composition vectors of all genes within the same genome for storage, as shown in Figure 3. Values for the same Kmer are stored according to gene numbers. Thus, when comparing two genomes, instead of performing $N_i \times N_j$ comparisons, only one comparison is needed. In practice, besides the data structure shown in Figure 3, CVArray also utilizes an Index file for each Kmer. Therefore, simply aligning the two Index files first allows calculations to be performed only on aligned Kmer dimensions.

In Figure 3, Kmers are sorted. Therefore, in the alignment operation for Kmer dimensions, our current algorithm aligns when keys are equal; if keys are unequal, it moves the sequence with the smaller key. The complexity of this algorithm is less than $d_i + d_j$ (d_i, d_j are the dimensionalities of the two CVs), and it is relatively simple to implement. When d_i and d_j are not significantly different, its complexity is lower than that of binary search

$(d_i \cdot \log_2 d_j)$. Theoretically, with an efficient hash algorithm, the complexity of aligning two CVs could be reduced to $\min(d_i, d_j)$. Cross-binary search could also achieve good efficiency, but these algorithms are more complex to implement and have higher meta-computational overhead, resulting in less ideal practical efficiency.

4 Object-Oriented Software Design

Figure 4 illustrates the basic design pattern of CVNet. Obtaining the connection network between genes via CVNet involves three steps: calculating the gene composition vector alignment matrix for each genome, constructing the similarity matrix based on the composition vector matrix, and deriving the connection network from the similarity matrix. Therefore, we designed three interfaces (C++ virtual classes): *CVmeth*, *SimilarMeth*, and *EdgeMeth*, to implement these three steps, respectively. Simultaneously, three calling functions are provided: *getcva*, *getMatrix*, and *getNet*. After setting parameters, the desired results can be obtained by simply calling the corresponding functions. Although member functions perform basic operations, each of these three classes contains a factory function named *create*, and virtual functions named *cv*, *calcOneK/_scale*, and *sm2edge*, respectively. Adding a new method within a step requires only two steps:

1. Derive a class from the base class and implement the computational algorithm in the derived class to override the virtual function in the base class.
2. Add an entry in the base class's factory function (named *create*) to provide the selection method.

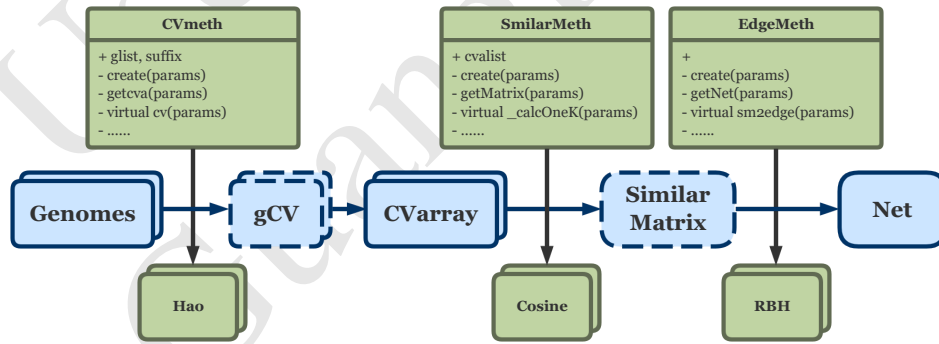


Figure 4: CVNet Object-Oriented Design

5 Basic Usage

The `cvnet` command is the core command of the software. It has a built-in automated workflow that can directly obtain phylogenetic trees from FASTA files. Additionally, a `dump` command is available for viewing compressed structures. Figure 5 shows the flowchart of the `cvnet` command. Methods based on high-dimensional modeling can construct heterogeneity matrices in gene networks and fill in missing heterogeneity between two genomes through specific algorithms. Users only need to prepare the genome FASTA files and a genome list, and set options according to project requirements.

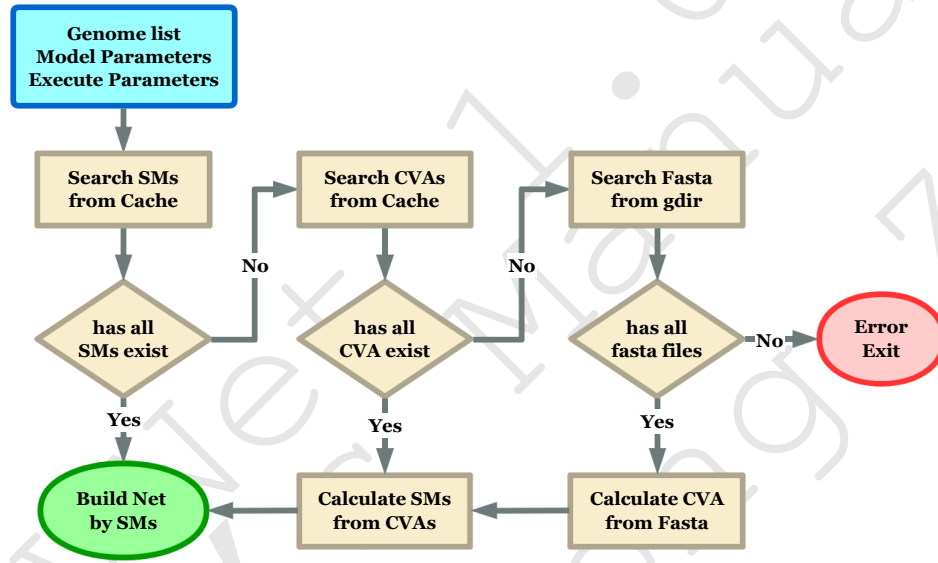


Figure 5: `cvnet` Command Flowchart

Common parameters for the `cvnet` command are shown in the table below. Parameters for other commands can be found in the software documentation.

Optional arguments:

<code>-h, --help</code>	shows help message and exits
<code>-i, --genome-file-list</code>	genome file list [default: "list"]
<code>-p, --pair-index</code>	Index for comparing genomes
<code>-L, --letter-group</code>	letter group for amino acid/nucleotide
<code>-g, --genome-type</code>	genome file type, faa/ffn [default: "faa"]
<code>-G, --gndir</code>	directory for genome file [default: ""]
<code>-v, --cv-method</code>	method for composition vector, Hao/Count [default: "Count"]
<code>-k, --kmer-length</code>	cutoff for kmer length [default: 5]
<code>-s, --similar-method</code>	method for similarity,

Cosine/InterList/InterSet/Jaccard/Dice
[default: "InterList"]

-e, --edge-method method for selecting edge,
RBH/GRB/CUT/SRB [default: "GRB"]

-c, --cutoff cutoff for edge similarity [default: 0.1]

-m, --mindist minimum for similarity [default: -0.1]

-C, --cache super directory for Cache files
[default: "cache"]

-o, --outfile output file name
[default: "faa.Count5.InterList.GRB10"]

-F, --out-format output file format: mcl/edge
[default: "mcl"]

-N, --index-file gene index file name
[default: "GeneIndex.tsv"]

-O, --outdir output directory [default: "mcl/"]

-B, --break-point stop program after obtained cva/sm/grp
[default: "None"]

-I, --mcl-inflate MCL inflation [default: 1.5]

-P, --mcl-pruning MCL pruning threshold [default: 1e-05]

-M, --mcl-iterations MCL maximum iterations [default: 100]

-q, --quiet run command in quiet mode

References

- Walter M. Fitch. Distinguishing homologous from analogous proteins. *Systematic Biology*, 19(2):99–113, June 1970. doi: 10.2307/2412448.
- Eugene V. Koonin. Orthologs, paralogs, and evolutionary Genomics1. *Annual Review of Genetics*, 39(Volume 39, 2005):309–338, December 2005. doi: 10.1146/annurev.genet.39.073003.114725.
- R. L. Tatusov, E. V. Koonin, and D. J. Lipman. A genomic perspective on protein families. *Science*, 278(5338):631–7, October 1997. doi: 10.1126/science.278.5338.631.
- Stephen F AltschuP, Warren Gish, Webb Miller, Eugene W Myers, and David J Lipman. Basic local alignment search tool. *Journal of Molecular Biology*, 215(3):403–410, October 1990. doi: 10.1016/S0022-2836(05)80360-2.

- Jonathan A. Eisen. Phylogenomics: Improving functional predictions for uncharacterized genes by evolutionary analysis. *Genome Research*, 8(3):163–167, January 1998. doi: 10.1101/gr.8.3.163.
- Christian M. Zmasek and Sean R. Eddy. A simple algorithm to infer gene duplication and speciation events on a gene tree. *Bioinformatics*, 17(9):821–828, September 2001. doi: 10.1093/bioinformatics/17.9.821.
- Li Li, Christian J. Stoeckert, and David S. Roos. OrthoMCL: Identification of ortholog groups for eukaryotic genomes. *Genome Research*, 13(9):2178–2189, September 2003. doi: 10.1101/gr.1224503.
- David M. Emms and Steven Kelly. OrthoFinder: Solving fundamental biases in whole genome comparisons dramatically improves orthogroup inference accuracy. *Genome Biology*, 16(1):157, August 2015. doi: 10.1186/s13059-015-0721-2.
- Guanghong Zuo. CVTree: A parallel alignment-free phylogeny and taxonomy tool based on composition vectors of genomes. *Genomics, Proteomics and Bioinformatics*, 19(4): 662–667, August 2021. doi: 10.1016/j.gpb.2021.03.006.
- Yi-Fei Lu, Xiao-Yang Zhi, and Guang-Hong Zuo. CVTree for 16S rRNA: Constructing taxonomy-compatible all-species living tree effectively and efficiently. *Chinese Physics B*, 34(8):088704, July 2025. doi: 10.1088/1674-1056/add508.
- J. Qi, B. Wang, and B. Hao. Whole proteome prokaryote phylogeny without sequence alignment: A K-string composition approach. *J Mol Evol*, 58(1):1–11, January 2004a. doi: 10.1007/s00239-003-2493-7.
- J. Qi, H. Luo, and B. Hao. CVTree: A phylogenetic tree reconstruction tool based on whole genomes. *Nucleic Acids Res*, 32(Web Server issue):W45–7, July 2004b. doi: 10.1093/nar/gkh362.
- Z. Xu and B. Hao. CVTree update: A newly designed phylogenetic study platform using composition vectors and whole genomes. *Nucleic Acids Res*, 37(Web Server issue):W174–8, July 2009. doi: 10.1093/nar/gkp278.
- G. Zuo and B. Hao. CVTree3 web server for whole-genome-based and alignment-free prokaryotic phylogeny and taxonomy. *Genomics Proteomics Bioinformatics*, 13(5):321–31, October 2015. doi: 10.1016/j.gpb.2015.08.004.